1. Параметры VM и модель GPU

CPU, ядра, потоки

```
devchatay@epds73cmval2ppf6qhpm:~$ lscpu
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          40 bits physical, 57 bits virtual
  Byte Order:             Little Endian
CPU(s):                   8
  On-line CPU(s) list:    0-7
Vendor ID:                GenuineIntel
  Model name:             Intel Xeon Processor (Icelake)
    CPU family:           6
    Model:                106
    Thread(s) per core:   2
    Core(s) per socket:   4
    Socket(s):            1
    Stepping:             0
    BogoMIPS:             4000.00
    Flags:                fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fx
                          sr sse sse2 ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology nonst
                          op_tsc cpuid tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic mov
                          be popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
                          fetch cpuid_fault invpcid_single ssbd ibrs ibpb stibp ibrs_enhanced fsgsbase bmi1 avx2
                           smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb av
                          x512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 wbnoinvd arat avx512vbmi umip
                          pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq
                           la57 rdpid fsrm md_clear arch_capabilities
Virtualization features:
  Hypervisor vendor:      KVM
  Virtualization type:    full
Caches (sum of all):
  L1d:                    256 KiB (8 instances)
  L1i:                    256 KiB (8 instances)
  L2:                     16 MiB (4 instances)
  L3:                     16 MiB (1 instance)
NUMA:
  NUMA node(s):           1
  NUMA node0 CPU(s):      0-7
Vulnerabilities:
  Gather data sampling:   Not affected
  Indirect target selection: Mitigation; Aligned branch/return thunks
  Itlb multihit:          Not affected
  L1tf:                   Not affected
  Mds:                    Not affected
  Meltdown:               Not affected
  Mmio stale data:        Mitigation; Clear CPU buffers; SMT Host state unknown
  Reg file data sampling: Not affected
  Retbleed:               Not affected
  Spec rstack overflow:   Not affected
  Spec store bypass:      Mitigation; Speculative Store Bypass disabled via prctl and seccomp
  Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:             Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS SW sequence; BHI
                          SW loop, KVM SW loop
  Srbds:                  Not affected
  Tsa:                    Not affected
  Tsx async abort:        Not affected
```

RAM

```
devchatay@epds73cmval2ppf6qhpm:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:            31Gi       3.2Gi        13Gi       106Mi        14Gi        27Gi
```

Swap: 0B 0B 0B

Модель GPU

devchatay@epds73cmval2ppf6qhpm:~$ lspci | grep -i nvidia
8b:00.0 3D controller: NVIDIA Corporation TU104GL [Tesla T4] (rev a1)

Тип VM

devchatay@epds73cmval2ppf6qhpm:~$ sudo dmidecode -t system
dmidecode 3.3
Getting SMBIOS data from sysfs.
SMBIOS 2.8 present.
Handle 0x0100, DMI type 1, 27 bytes
System Information
Manufacturer: Yandex
Product Name: xeon-gold-6338
Version: pc-q35-yc-5.0
Serial Number: YC-epds73cmval2ppf6qhpm
UUID: 23000007-65bc-38d9-6faa-a2ce5e6d4736
Wake-up Type: Power Switch
SKU Number: Not Specified
Family: Yandex Cloud
Handle 0x2000, DMI type 32, 11 bytes
System Boot Information
Status: No errors detected

2. Версии Ubuntu, Kubernetes, GPU Operator, драйвера NVIDIA и CUDA runtime

Версия Ubuntu

devchatay@epds73cmval2ppf6qhpm:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 22.04.5 LTS
Release: 22.04
Codename: jammy

Версия Kubernetes

devchatay@epds73cmval2ppf6qhpm:~$ kubectl version --client
Client Version: v1.34.8
Kustomize Version: v5.7.1

devchatay@epds73cmval2ppf6qhpm:~$ kubectl version
Client Version: v1.34.8
Kustomize Version: v5.7.1
Server Version: v1.34.8

Версия NVIDIA GPU Operator

devchatay@epds73cmval2ppf6qhpm:~$ kubectl get deployment gpu-operator -n gpu-operator -o jsonpath='{.spec.template.spec.containers[0].image}'

nvcr.io/nvidia/gpu-operator:v25.10.1

devchatay@epds73cmval2ppf6qhpm:~$ kubectl get pods -n gpu-operator -l app=gpu-operator -o jsonpath='{.items[0].spec.containers[0].image}'
nvcr.io/nvidia/gpu-operator:v25.10.1

Версия драйвера NVIDIA

devchatay@epds73cmval2ppf6qhpm:~$ cat /proc/driver/nvidia/version | head -n 1
NVRM version: NVIDIA UNIX Open Kernel Module for x86_64 580.105.08 Release Build (dvs-builder@U22-I3-B10-02-5) Wed Oct 29 22:29:53 UTC 2025

Версия CUDA Runtime

devchatay@epds73cmval2ppf6qhpm:~$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Thu_Nov_18_09:45:30_PST_2021
Cuda compilation tools, release 11.5, V11.5.119
Build cuda_11.5.r11.5/compiler.30672275_0

3. Скриншот или текстовый вывод kubectl get nodes -o wide

devchatay@epds73cmval2ppf6qhpm:~$ kubectl get nodes -o wide;
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME
epds73cmval2ppf6qhpm Ready control-plane 5d21h v1.34.8 10.129.0.25 <none> Ubuntu 22.04.5 LTS 5.15.0-157-generic containerd://2.2.1

4. Скриншот или текстовый вывод kubectl get pods -A

devchatay@epds73cmval2ppf6qhpm:~$ kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
gpu-lab torch-training-exclusive-wdwzk 0/1 Completed 0 27m
gpu-operator gpu-feature-discovery-dxjbl 2/2 Running 0 4d19h
gpu-operator gpu-operator-7569f8b499-5ck8q 1/1 Running 9 (4h8m ago) 4d19h
gpu-operator gpu-operator-node-feature-discovery-gc-55ffc49ccc-btq9c 1/1 Running 0 4d19h
gpu-operator gpu-operator-node-feature-discovery-master-6b5787f695-pw6z6 1/1 Running 6 (4h8m ago) 4d19h
gpu-operator gpu-operator-node-feature-discovery-worker-x9m5g 1/1 Running 0 4d19h
gpu-operator nvidia-container-toolkit-daemonset-xzb7f 1/1 Running 0 4d19h
gpu-operator nvidia-cuda-validator-cbbch 0/1 Completed 0 4d19h
gpu-operator nvidia-dcgm-exporter-qtfrc 1/1 Running 0 14h
gpu-operator nvidia-device-plugin-daemonset-qhpfx 2/2 Running 0 31m
gpu-operator nvidia-driver-daemonset-gvv8v 1/1 Running 0 4d19h
gpu-operator nvidia-operator-validator-cnmkn 1/1 Running 0 4d19h
kube-flannel kube-flannel-ds-mgb79 1/1 Running 0 5d21h
kube-system coredns-66bc5c9577-87hsd 1/1 Running 0 5d21h
kube-system coredns-66bc5c9577-f5xsv 1/1 Running 0 5d21h

```
kube-system    etcd-epds73cmval2ppf6qhpm                              1/1     Running     0              5d21h
kube-system    kube-apiserver-epds73cmval2ppf6qhpm                    1/1     Running     0              5d21h
kube-system    kube-controller-manager-epds73cmval2ppf6qhpm           1/1     Running     8 (4h21m ago)   5d21h
kube-system    kube-proxy-lvjvx                                       1/1     Running     0              5d21h
kube-system    kube-scheduler-epds73cmval2ppf6qhpm                    1/1     Running     8 (4h8m ago)    5d21h
monitoring     alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running     0              4d18h
monitoring     monitoring-grafana-6b866c5c-jzl64                      3/3     Running     0              4d18h
monitoring     monitoring-kube-prometheus-operator-554ffdbcd5-9plls   1/1     Running     0              28m
monitoring     monitoring-kube-state-metrics-7f7f8474d9-vnk8f         1/1     Running     1 (5h ago)     4d18h
monitoring     monitoring-prometheus-node-exporter-gnjzc              1/1     Running     0              4d18h
monitoring     prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running     0              14h
```

5. Вывод nvidia-smi из pod;

Манифест для запуска пода и исполнения nvidia-smi

```
devchatay@epds73cmval2ppf6qhpm:~$ cat > ~/manifests/torch-training-testnvidiasmi.yaml << 'EOF'
apiVersion: batch/v1
kind: Job
metadata:
  name: nvidia-smi-test
  namespace: gpu-lab
spec:
  ttlSecondsAfterFinished: 3600
  backoffLimit: 0
  template:
    metadata:
      labels:
        app: nvidia-smi-test
    spec:
      restartPolicy: Never
      containers:
        - name: smi
          image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
          imagePullPolicy: IfNotPresent
          command: ["/bin/bash", "-c"]
          args:
            - |
              echo "=== NVIDIA-SMI ==="
              nvidia-smi
              echo ""
              echo "=== CUDA Runtime (nvcc) ==="
              nvcc --version 2>/dev/null || echo "nvcc not found in this image"
EOF          nvidia.com/gpu: "1" n:', torch.backends.cudnn.version())" 2>/dev/null || echo "PyTorch not available".cuda.is_available());
print('CUDA version (compiled with):', torch.
cat > ~/manifests/torch-training-testnvidiasmi.yaml << 'EOF'
apiVersion: batch/v1
kind: Job
metadata:
  name: nvidia-smi-test
  namespace: gpu-lab
spec:
  ttlSecondsAfterFinished: 3600
  backoffLimit: 0
  template:
    metadata:
      labels:
        app: nvidia-smi-test
    spec:
```

```
    restartPolicy: Never
    containers:
      - name: smi
        image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
        imagePullPolicy: IfNotPresent
        command: ["/bin/bash", "-c"]
        args:
          - |
            echo "=== NVIDIA-SMI ==="
            nvidia-smi
            echo ""
            echo "=== CUDA Runtime (nvcc) ==="
            nvcc --version 2>/dev/null || echo "nvcc not found in this image"
            echo ""
            echo "=== PyTorch CUDA info ==="
            python -c "import torch; print('PyTorch version:', torch.__version__); print('CUDA available:', torch.cuda.is_available());
print('CUDA version (compiled with):', torch.version.cuda); print('cuDNN version:', torch.backends.cudnn.version())" 2>/dev/null || echo
"PyTorch not available"
        resources:
          limits:
            nvidia.com/gpu: "1"
          requests:
            nvidia.com/gpu: "1"
EOF
```

Исполнение манифеста и результат в логах:

```
devchatay@epds73cmval2ppf6qhpm:~$ kubectl apply -f ~/manifests/torch-training-testnvidiasmi.yaml
job.batch/nvidia-smi-test created
devchatay@epds73cmval2ppf6qhpm:~$ kubectl logs -n gpu-lab job/nvidia-smi-test
=== NVIDIA-SMI ===
Sun May 31 12:34:31 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.105.08             Driver Version: 580.105.08     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       On  |   00000000:8B:00.0 Off |                    0 |
| N/A   35C    P8             14W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+

=== CUDA Runtime (nvcc) ===
nvcc not found in this image

=== PyTorch CUDA info ===
PyTorch version: 2.5.1+cu124
CUDA available: True
CUDA version (compiled with): 12.4
cuDNN version: 90100
devchatay@epds73cmval2ppf6qhpm:~$
```

Выполнялось в состоянии, когда другие процессы (другие нагрузки на GPU, обучения) не исполнялись

6. **Результаты benchmark для exclusive, time-slicing и, если получилось, MPS;**

В качестве задачи сначала было взято дообучение (fine-tuning) LLM. Пример exclusive yaml-файла:

```
cat > ~/manifests/torch-training-shared.yaml << 'EOF'
apiVersion: batch/v1
kind: Job
metadata:
  name: torch-training-shared
  namespace: gpu-lab
spec:
  ttlSecondsAfterFinished: 3600
  backoffLimit: 0
  template:
    metadata:
      labels:
        app: torch-training
        mode: shared
    spec:
      restartPolicy: Never
      containers:
        - name: trainer
          image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
          imagePullPolicy: IfNotPresent
          command:
            - /bin/bash
            - -c
            - |
              set -e
              echo "Installing dependencies..."
              pip install --no-cache-dir --quiet transformers==4.40.0 accelerate==0.29.0 datasets==2.19.0

              echo "Starting training..."
              python -c "
              import os, time, json, torch
              from transformers import AutoTokenizer, AutoModelForCausalLM, TrainingArguments, Trainer, TrainerCallback
              import torch.utils.data

              step_times, peak_mem_gb = [], 0
              last_t = None

              def log_gpu_info():
                  if torch.cuda.is_available():
                      name = torch.cuda.get_device_name(0)
                      total_mem = torch.cuda.get_device_properties(0).total_memory / 1024**3
                      print(f'GPU: {name}')
                      print(f'TOTAL_GPU_MEMORY_GB: {total_mem:.2f}')
                      return name, total_mem
                  return 'Unknown', 0

              class MetricsCallback(TrainerCallback):
                  def on_step_begin(self, args, state, control, **kwargs):
                      global last_t
                      last_t = time.time()
                      torch.cuda.reset_peak_memory_stats()
                  def on_step_end(self, args, state, control, **kwargs):
                      global peak_mem_gb
                      if last_t:
                          step_times.append(time.time() - last_t)
                          peak_mem_gb = max(peak_mem_gb, torch.cuda.max_memory_allocated() / 1024**3)
                      if state.global_step % 5 == 0:
                          print(f'Step {state.global_step}: time={step_times[-1]:.3f}s, peak_mem={peak_mem_gb:.2f}GB')
                  def on_train_end(self, args, state, control, **kwargs):
                      if step_times:
                          avg, mn, mx = sum(step_times)/len(step_times), min(step_times), max(step_times)
                          print('\n' + '='*60)
                          print(f'avg_step_sec: {avg:.4f}')
                          print(f'min_step_sec: {mn:.4f}')
                          print(f'max_step_sec: {mx:.4f}')
                          print(f'peak_cuda_mem_gb: {peak_mem_gb:.2f}')
                          print(f'METRICS_JSON: {json.dumps({\"avg_step_sec\":round(avg,4),\"min_step_sec\":round(mn,4),\"max_step_sec\":round(mx,4),\"peak_cuda_mem_gb\":round(peak_mem_gb,2)})}')
                          print('='*60 + '\n')

              gpu_name, total_mem = log_gpu_info()
              model_name = 'Qwen/Qwen2.5-0.5B-Instruct'
              hf_token = os.getenv('HF_TOKEN', '')
```

```
                    peak_mem_gb = max(peak_mem_gb, torch.cuda.max_memory_allocated() / 1024**3)
                if state.global_step % 5 == 0:
                    print(f'Step {state.global_step}: time={step_times[-1]:.3f}s, peak_mem={peak_mem_gb:.2f}GB')
            def on_train_end(self, args, state, control, **kwargs):
                if step_times:
                    avg, mn, mx = sum(step_times)/len(step_times), min(step_times), max(step_times)
                    print('\n' + '='*60)
                    print(f'avg_step_sec: {avg:.4f}')
                    print(f'min_step_sec: {mn:.4f}')
                    print(f'max_step_sec: {mx:.4f}')
                    print(f'peak_cuda_mem_gb: {peak_mem_gb:.2f}')
                    print(f'METRICS_JSON: {json.dumps({\"avg_step_sec\":round(avg,4),\"min_step_sec\":round(mn,4),\"max_step_sec\":round(mx,4),\"peak_cuda_mem_gb\":round(peak_mem_gb,2)})}')
                    print('='*60 + '\n')

        gpu_name, total_mem = log_gpu_info()
        model_name = 'Qwen/Qwen2.5-0.5B-Instruct'
        hf_token = os.getenv('HF_TOKEN', '')

        tokenizer = AutoTokenizer.from_pretrained(model_name, token=hf_token)
        model = AutoModelForCausalLM.from_pretrained(model_name, token=hf_token).cuda()

        train_texts = ['MOCK DATASET TEXT FOR MODEL TRAINING' * 5] * 500
        encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=64)

        class SimpleDataset(torch.utils.data.Dataset):
            def __init__(self, enc): self.enc = enc
            def __getitem__(self, idx):
                item = {k: torch.tensor(v[idx]) for k, v in self.enc.items()}
                item['labels'] = item['input_ids'].clone()
                item['labels'][item['labels'] == tokenizer.pad_token_id] = -100
                return item
            def __len__(self): return len(self.enc['input_ids'])

        training_args = TrainingArguments(
            output_dir='./results',
            num_train_epochs=3,
            per_device_train_batch_size=1,
            save_steps=10000,
            logging_steps=5,
            bf16=torch.cuda.is_bf16_supported(),
            report_to='none',
            gradient_accumulation_steps=2,
        )

        trainer = Trainer(
            model=model, args=training_args, train_dataset=SimpleDataset(encodings),
            callbacks=[MetricsCallback()]
        )

        print('Starting training...')
        t0 = time.time()
        trainer.train()
        print(f'Done in {time.time()-t0:.2f} sec')
        "
      env:
        - name: HF_TOKEN
          valueFrom:
            secretKeyRef:
              name: hf-token-secret
              key: HF_TOKEN
              optional: true
      resources:
        limits:
          cpu: "1"
          memory: 2Gi
          nvidia.com/gpu.shared: "1"
        requests:
          cpu: "1"
          memory: 1Gi
          nvidia.com/gpu.shared: "1"
EOF
```

cat > ~/manifests/torch-training-exclusive.yaml << 'EOF'

apiVersion: batch/v1

kind: Job

metadata:

  name: torch-training-exclusive

  namespace: gpu-lab

spec:

  ttlSecondsAfterFinished: 3600

  backoffLimit: 0

  template:

    metadata:

      labels:

        app: torch-training

        mode: exclusive

    spec:

      restartPolicy: Never

      containers:

        - name: trainer

          image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime

```
imagePullPolicy: IfNotPresent
command:
 - /bin/bash
 - -c
 - |
   set -e
   echo "Installing dependencies..."
   pip install --no-cache-dir --quiet transformers==4.40.0 accelerate==0.29.0

   echo "Starting training..."
   python -c "
   import os, time, json, torch, sys, gc
   from transformers import AutoTokenizer, AutoModelForCausalLM, TrainingArguments, Trainer, TrainerCallback
   import torch.utils.data

   step_times, peak_mem_gb = [], 0
   last_t = None

   def log_gpu_info():
       if torch.cuda.is_available():
           name = torch.cuda.get_device_name(0)
           total_mem = torch.cuda.get_device_properties(0).total_memory / 1024**3
           print(f'GPU: {name}')
           print(f'TOTAL_GPU_MEMORY_GB: {total_mem:.2f}')
           return name, total_mem
       return 'Unknown', 0

   class MetricsCallback(TrainerCallback):
       def on_step_begin(self, args, state, control, **kwargs):
           global last_t
           last_t = time.time()
           torch.cuda.reset_peak_memory_stats()
       def on_step_end(self, args, state, control, **kwargs):
```

```
            global peak_mem_gb
            if last_t:
                step_times.append(time.time() - last_t)
                peak_mem_gb = max(peak_mem_gb, torch.cuda.max_memory_allocated() / 1024**3)
            if state.global_step % 2 == 0:
                print(f'Step {state.global_step}: time={step_times[-1]:.3f}s, peak_mem={peak_mem_gb:.2f}GB')
                sys.stdout.flush()
            # Очистка кэша после каждого шага для экономии памяти
            if state.global_step % 5 == 0:
                torch.cuda.empty_cache()
                gc.collect()
        def on_train_end(self, args, state, control, **kwargs):
            if step_times:
                avg, mn, mx = sum(step_times)/len(step_times), min(step_times), max(step_times)
                print('\n' + '='*60)
                print(f'avg_step_sec: {avg:.4f}')
                print(f'min_step_sec: {mn:.4f}')
                print(f'max_step_sec: {mx:.4f}')
                print(f'peak_cuda_mem_gb: {peak_mem_gb:.2f}')
                print(f'METRICS_JSON:
{json.dumps({\"avg_step_sec\":round(avg,4),\"min_step_sec\":round(mn,4),\"max_step_sec\":round(mx,4),\"peak_cuda_mem_gb\":round(p
eak_mem_gb,2)})}')
                print('='*60 + '\n')
                sys.stdout.flush()

    gpu_name, total_mem = log_gpu_info()

    model_name = 'Qwen/Qwen2.5-0.5B-Instruct'
    hf_token = os.getenv('HF_TOKEN', '')

    tokenizer = AutoTokenizer.from_pretrained(model_name, token=hf_token)
    model = AutoModelForCausalLM.from_pretrained(model_name, token=hf_token).cuda()

    train_texts = ['GPU lab training text' * 3] * 100
```

```
encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=32)

class SimpleDataset(torch.utils.data.Dataset):
    def __init__(self, enc): self.enc = enc
    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.enc.items()}
        item['labels'] = item['input_ids'].clone()
        item['labels'][item['labels'] == tokenizer.pad_token_id] = -100
        return item
    def __len__(self): return len(self.enc['input_ids'])

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=1,
    per_device_train_batch_size=1,
    save_steps=10000,
    logging_steps=2,
    bf16=torch.cuda.is_bf16_supported(),
    report_to='none',
    optim='adafactor',
    gradient_checkpointing=True,
    dataloader_num_workers=0,
)

trainer = Trainer(
    model=model, args=training_args, train_dataset=SimpleDataset(encodings),
    callbacks=[MetricsCallback()]
)

print('Starting training...')
print('TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!')
sys.stdout.flush()
t0 = time.time()
```

```
          trainer.train()
          print(f'Done in {time.time()-t0:.2f} sec')
          "
        env:
         - name: HF_TOKEN
           valueFrom:
             secretKeyRef:
               name: hf-token-secret
               key: HF_TOKEN
               optional: true
        resources:
          limits:
            cpu: "2"
            memory: 4Gi
            nvidia.com/gpu: "1"
          requests:
            cpu: "1"
            memory: 2Gi
            nvidia.com/gpu: "1"
EOF
```

Но возникли ограничения с shared - 4 пода не влезали по памяти на 1 Tesla T4.

Была взята другая задача - обучение на задачу классификации. Сеть состоит из линейных слоев, дропаутов и активаций. Чтобы сетап оправдывал использование GPU и был CPU тяжело исполняемым, взяла большой hidden_dim, batch_size. Таким образом увеличиваем нагрузку на память и делаем больше параллельных вычислений, на что как раз таки рассчитано GPU. Чтобы задача исполнялась не слишком быстро, взято много данных и поставлено большое количество эпох.

Файлы итоговых манифестов:

EXCLUSIVE

```
devchatay@epds73cmval2ppf6qhpm:~/manifests$ cat torch-training-exclusive.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: torch-training-exclusive
  namespace: gpu-lab
spec:
  ttlSecondsAfterFinished: 3600
  backoffLimit: 0
  template:
    metadata:
      labels:
        app: torch-training
        mode: exclusive
    spec:
      restartPolicy: Never
      containers:
        - name: trainer
          image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
          imagePullPolicy: IfNotPresent
          command:
            - /bin/bash
            - -c
            - |
              set -e
              echo "Installing dependencies..."
              pip install --no-cache-dir --quiet transformers==4.40.0 accelerate==0.29.0

              echo "Starting training..."
              python -c "
              import os, time, json, torch, sys, gc
              import torch.nn as nn
              import torch.utils.data
              from transformers import TrainerCallback
              step_times, peak_mem_gb = [], 0
              last_t = None
              def log_gpu_info():
                  if torch.cuda.is_available():
                      name = torch.cuda.get_device_name(0)
                      total_mem = torch.cuda.get_device_properties(0).total_memory / 1024**3
                      print(f'GPU: {name}')
                      print(f'TOTAL_GPU_MEMORY_GB: {total_mem:.2f}')
                      return name, total_mem
                  return 'Unknown', 0
              class MetricsCallback(TrainerCallback):
                  def on_step_begin(self, args, state, control, **kwargs):
                      global last_t
                      last_t = time.time()
                      torch.cuda.reset_peak_memory_stats()
                  def on_step_end(self, args, state, control, **kwargs):
                      global peak_mem_gb
                      if last_t:
                          step_times.append(time.time() - last_t)
                          peak_mem_gb = max(peak_mem_gb, torch.cuda.max_memory_allocated() / 1024**3)
                      if state.global_step % 2 == 0:
                          print(f'Step {state.global_step}: time={step_times[-1]:.3f}s, peak_mem={peak_mem_gb:.2f}GB')
                          sys.stdout.flush()
                      if state.global_step % 5 == 0:
                          torch.cuda.empty_cache()
                          gc.collect()
                  def on_train_end(self, args, state, control, **kwargs):
                      if step_times:
                          avg, mn, mx = sum(step_times)/len(step_times), min(step_times), max(step_times)
                          print('\n' + '='*60)
                          print(f'avg_step_sec: {avg:.4f}')
                          print(f'min_step_sec: {mn:.4f}')
```

```
                print(f'max_step_sec: {mx:.4f}')
                print(f'peak_cuda_mem_gb: {peak_mem_gb:.2f}')
                print(f'METRICS_JSON:
{json.dumps({\"avg_step_sec\":round(avg,4),\"min_step_sec\":round(mn,4),\"max_step_sec\":round(mx,4),\"peak_cuda_mem_gb\":round(p
eak_mem_gb,2)})}')
                print('='*60 + '\n')
                sys.stdout.flush()
        class MockState:
            global_step = 0
        class MockArgs: pass
        class MockControl: pass
        gpu_name, total_mem = log_gpu_info()

        # Синтетический датасет для классификации
        N_SAMPLES = 100000
        INPUT_DIM = 256
        HIDDEN_DIM = 4096 # Большие слои для нагрузки на память
        NUM_CLASSES = 10
        BATCH_SIZE = 16384 # большой батч для нагрузки на память
        NUM_EPOCHS = 50
        torch.manual_seed(42)
        X = torch.randn(N_SAMPLES, INPUT_DIM).float()
        y = torch.argmax(X[:, :NUM_CLASSES] + torch.randn(N_SAMPLES, NUM_CLASSES) * 0.3, dim=1)
        class SyntheticDataset(torch.utils.data.Dataset):
            def __init__(self, X, y):
                self.X = X
                self.y = y
            def __getitem__(self, idx):
                return {'input_ids': self.X[idx], 'labels': self.y[idx]}
            def __len__(self): return len(self.X)
        model = nn.Sequential(
            nn.Linear(INPUT_DIM, HIDDEN_DIM),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(HIDDEN_DIM, HIDDEN_DIM),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(HIDDEN_DIM, HIDDEN_DIM),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(HIDDEN_DIM, HIDDEN_DIM // 2),
            nn.ReLU(),
            nn.Linear(HIDDEN_DIM // 2, NUM_CLASSES)
        ).cuda()
        device = next(model.parameters()).device
        print(device)
        total_params = sum(p.numel() for p in model.parameters())
        print(f'Всего параметров: {total_params}')
        trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
        print(f'Обучаемых параметров: {trainable_params}')
        criterion = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
        dataset = SyntheticDataset(X, y)
        metrics = MetricsCallback()
        state = MockState()
        args = MockArgs()
        control = MockControl()
        print('Starting training...')
        print('TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!')
        sys.stdout.flush()
        t0 = time.time()
        for epoch in range(NUM_EPOCHS):
            model.train()
            for i in range(0, len(dataset), BATCH_SIZE):
                metrics.on_step_begin(args, state, control)
```

```
                end_idx = min(i + BATCH_SIZE, len(dataset))
                batch_X = dataset.X[i:end_idx].cuda()
                batch_y = dataset.y[i:end_idx].cuda()

                optimizer.zero_grad()
                output = model(batch_X)
                loss = criterion(output, batch_y)
                loss.backward()
                optimizer.step()

                state.global_step += 1
                metrics.on_step_end(args, state, control)

            if epoch % 5 == 0:
                torch.cuda.empty_cache()
                gc.collect()
        metrics.on_train_end(args, state, control)
        print(f'Done in {time.time()-t0:.2f} sec')
        "
      env:
        - name: HF_TOKEN
          valueFrom:
            secretKeyRef:
              name: hf-token-secret
              key: HF_TOKEN
              optional: true
      resources:
        limits:
          cpu: "2"
          memory: 4Gi
          nvidia.com/gpu: "1"
        requests:
          cpu: "1"
          memory: 2Gi
          nvidia.com/gpu: "1"
```

SHARED

```
devchatay@epds73cmval2ppf6qhpm:~/manifests$ cat torch-training-shared.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: torch-training-shared
  namespace: gpu-lab
spec:
  ttlSecondsAfterFinished: 3600
  backoffLimit: 0
  template:
    metadata:
      labels:
        app: torch-training
        mode: shared
    spec:
      restartPolicy: Never
      containers:
        - name: trainer
          image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
          imagePullPolicy: IfNotPresent
          command:
            - /bin/bash
            - -c
```

```
- |
  set -e
  echo "Installing dependencies..."
  pip install --no-cache-dir --quiet transformers==4.40.0 accelerate==0.29.0

  echo "Starting training..."
  python -c "
  import os, time, json, torch, sys, gc
  import torch.nn as nn
  import torch.utils.data
  from transformers import TrainerCallback
  step_times, peak_mem_gb = [], 0
  last_t = None
  def log_gpu_info():
      if torch.cuda.is_available():
          name = torch.cuda.get_device_name(0)
          total_mem = torch.cuda.get_device_properties(0).total_memory / 1024**3
          print(f'GPU: {name}')
          print(f'TOTAL_GPU_MEMORY_GB: {total_mem:.2f}')
          return name, total_mem
      return 'Unknown', 0
  class MetricsCallback(TrainerCallback):
      def on_step_begin(self, args, state, control, **kwargs):
          global last_t
          last_t = time.time()
          torch.cuda.reset_peak_memory_stats()
      def on_step_end(self, args, state, control, **kwargs):
          global peak_mem_gb
          if last_t:
              step_times.append(time.time() - last_t)
              peak_mem_gb = max(peak_mem_gb, torch.cuda.max_memory_allocated() / 1024**3)
          if state.global_step % 2 == 0:
              print(f'Step {state.global_step}: time={step_times[-1]:.3f}s, peak_mem={peak_mem_gb:.2f}GB')
              sys.stdout.flush()
          if state.global_step % 5 == 0:
              torch.cuda.empty_cache()
              gc.collect()
      def on_train_end(self, args, state, control, **kwargs):
          if step_times:
              avg, mn, mx = sum(step_times)/len(step_times), min(step_times), max(step_times)
              print('\n' + '='*60)
              print(f'avg_step_sec: {avg:.4f}')
              print(f'min_step_sec: {mn:.4f}')
              print(f'max_step_sec: {mx:.4f}')
              print(f'peak_cuda_mem_gb: {peak_mem_gb:.2f}')
              print(f'METRICS_JSON:
{json.dumps({\"avg_step_sec\":round(avg,4),\"min_step_sec\":round(mn,4),\"max_step_sec\":round(mx,4),\"peak_cuda_mem_gb\":round(p
eak_mem_gb,2)})}')
              print('='*60 + '\n')
              sys.stdout.flush()
  class MockState:
      global_step = 0
  class MockArgs: pass
  class MockControl: pass
  gpu_name, total_mem = log_gpu_info()

  # Синтетический датасет для классификации
  N_SAMPLES = 100000
  INPUT_DIM = 256
  HIDDEN_DIM = 4096  # Большие слои для нагрузки на память
  NUM_CLASSES = 10
  BATCH_SIZE = 16384 # большой батч для нагрузки на память
  NUM_EPOCHS = 50
  torch.manual_seed(42)
  X = torch.randn(N_SAMPLES, INPUT_DIM).float()
  y = torch.argmax(X[:, :NUM_CLASSES] + torch.randn(N_SAMPLES, NUM_CLASSES) * 0.3, dim=1)
```

```
    class SyntheticDataset(torch.utils.data.Dataset):
        def __init__(self, X, y):
            self.X = X
            self.y = y
        def __getitem__(self, idx):
            return {'input_ids': self.X[idx], 'labels': self.y[idx]}
        def __len__(self): return len(self.X)
    model = nn.Sequential(
        nn.Linear(INPUT_DIM, HIDDEN_DIM),
        nn.ReLU(),
        nn.Dropout(0.1),
        nn.Linear(HIDDEN_DIM, HIDDEN_DIM),
        nn.ReLU(),
        nn.Dropout(0.1),
        nn.Linear(HIDDEN_DIM, HIDDEN_DIM),
        nn.ReLU(),
        nn.Dropout(0.1),
        nn.Linear(HIDDEN_DIM, HIDDEN_DIM // 2),
        nn.ReLU(),
        nn.Linear(HIDDEN_DIM // 2, NUM_CLASSES)
    ).cuda()

    total_params = sum(p.numel() for p in model.parameters())
    print(f'Всего параметров: {total_params}')
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f'Обучаемых параметров: {trainable_params}')
    device = next(model.parameters()).device
    print(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    dataset = SyntheticDataset(X, y)
    metrics = MetricsCallback()
    state = MockState()
    args = MockArgs()
    control = MockControl()
    print('Starting training...')
    print('TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!')
    sys.stdout.flush()
    t0 = time.time()
    for epoch in range(NUM_EPOCHS):
        model.train()
        for i in range(0, len(dataset), BATCH_SIZE):
            metrics.on_step_begin(args, state, control)

            end_idx = min(i + BATCH_SIZE, len(dataset))
            batch_X = dataset.X[i:end_idx].cuda()
            batch_y = dataset.y[i:end_idx].cuda()

            optimizer.zero_grad()
            output = model(batch_X)
            loss = criterion(output, batch_y)
            loss.backward()
            optimizer.step()

            state.global_step += 1
            metrics.on_step_end(args, state, control)

        if epoch % 5 == 0:
            torch.cuda.empty_cache()
            gc.collect()
    metrics.on_train_end(args, state, control)
    print(f'Done in {time.time()-t0:.2f} sec')
    "
env:
  - name: HF_TOKEN
    valueFrom:
```

```
        secretKeyRef:
          name: hf-token-secret
          key: HF_TOKEN
          optional: true
    resources:
      limits:
        cpu: "1"
        memory: 2Gi
        nvidia.com/gpu.shared: "1"
      requests:
        cpu: "1"
        memory: 1Gi
        nvidia.com/gpu.shared: "1"
```

Логи исполнения:

EXCLUSIVE

```
devchatay@epds73cmval2ppf6qhpm:~$ kubectl logs -n gpu-lab -f job/torch-training-exclusive
Installing dependencies...
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPSConnectionPool(host='pypi.org', port=443): Read timed out. (read timeout=15)")': /simple/transformers/
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPSConnectionPool(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/09/c8/844d5518a6aeb4ffdc0cf0cae65ae13dbe5838306728c5c640b5a6e2a0c9/transformers-4.40.0-py3-none-any.whl.metadata
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Starting training...
GPU: Tesla T4
TOTAL_GPU_MEMORY_GB: 14.56
cuda:0
Всего параметров: 43026442
Обучаемых параметров: 43026442
Starting training...
TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!
Step 2: time=0.010s, peak_mem=2.63GB
Step 4: time=0.923s, peak_mem=2.63GB
Step 6: time=0.006s, peak_mem=2.63GB
Step 8: time=0.006s, peak_mem=2.63GB
Step 10: time=0.877s, peak_mem=2.63GB
….
Step 338: time=0.991s, peak_mem=2.63GB
Step 340: time=0.988s, peak_mem=2.63GB
Step 342: time=0.999s, peak_mem=2.63GB
Step 344: time=0.111s, peak_mem=2.63GB
Step 346: time=0.006s, peak_mem=2.63GB
Step 348: time=0.988s, peak_mem=2.63GB
Step 350: time=0.999s, peak_mem=2.63GB
============================================================
avg_step_sec: 0.6763
min_step_sec: 0.0027
max_step_sec: 1.0909
peak_cuda_mem_gb: 2.63
METRICS_JSON: {"avg_step_sec": 0.6763, "min_step_sec": 0.0027, "max_step_sec": 1.0909, "peak_cuda_mem_gb": 2.63}
============================================================
Done in 303.72 sec
devchatay@epds73cmval2ppf6qhpm:~$
```

SHARED

devchatay@epds73cmval2ppf6qhpm:~$ for i in 1 2 3 4; do echo "===== torch-benchmark-ts-$i ====="; kubectl logs -n gpu-lab "job/torch-training-ts-$i"; done
===== torch-benchmark-ts-1 =====
Installing dependencies...
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Starting training...
GPU: Tesla T4
TOTAL_GPU_MEMORY_GB: 14.56
Всего параметров: 43026442
Обучаемых параметров: 43026442
cuda:0
Starting training...
TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!
Step 2: time=0.018s, peak_mem=2.63GB
Step 4: time=1.966s, peak_mem=2.63GB
Step 6: time=0.006s, peak_mem=2.63GB
Step 8: time=0.006s, peak_mem=2.63GB
Step 10: time=2.508s, peak_mem=2.63GB
….
Step 342: time=4.257s, peak_mem=2.63GB
Step 344: time=0.530s, peak_mem=2.63GB
Step 346: time=0.018s, peak_mem=2.63GB
Step 348: time=4.274s, peak_mem=2.63GB
Step 350: time=4.212s, peak_mem=2.63GB
==
avg_step_sec: 2.9156
min_step_sec: 0.0044
max_step_sec: 4.3580
peak_cuda_mem_gb: 2.63
METRICS_JSON: {"avg_step_sec": 2.9156, "min_step_sec": 0.0044, "max_step_sec": 4.358, "peak_cuda_mem_gb": 2.63}
==
Done in 1289.25 sec
===== torch-benchmark-ts-2 =====
Installing dependencies...
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPSConnectionPool(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/09/c8/844d5518a6aeb4ffdc0cf0cae65ae13dbe5838306728c5c640b5a6e2a0c9/transformers-4.40.0-py3-none-any.whl.metadata
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Starting training...
GPU: Tesla T4
TOTAL_GPU_MEMORY_GB: 14.56
Всего параметров: 43026442
Обучаемых параметров: 43026442
cuda:0
Starting training...
TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!
Step 2: time=0.031s, peak_mem=2.63GB
Step 4: time=3.932s, peak_mem=2.63GB
Step 6: time=0.007s, peak_mem=2.63GB
Step 8: time=0.006s, peak_mem=2.63GB
Step 10: time=3.941s, peak_mem=2.63GB
….
Step 342: time=2.793s, peak_mem=2.63GB
Step 344: time=0.253s, peak_mem=2.63GB
Step 346: time=0.017s, peak_mem=2.63GB

Step 348: time=2.147s, peak_mem=2.63GB
Step 350: time=2.170s, peak_mem=2.63GB
==
avg_step_sec: 2.9169
min_step_sec: 0.0050
max_step_sec: 4.3611
peak_cuda_mem_gb: 2.63
METRICS_JSON: {"avg_step_sec": 2.9169, "min_step_sec": 0.005, "max_step_sec": 4.3611, "peak_cuda_mem_gb": 2.63}
==
Done in 1291.02 sec
===== torch-benchmark-ts-3 =====
Installing dependencies...
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Starting training...
GPU: Tesla T4
TOTAL_GPU_MEMORY_GB: 14.56
Всего параметров: 43026442
Обучаемых параметров: 43026442
cuda:0
Starting training...
TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!
Step 2: time=0.010s, peak_mem=2.63GB
Step 4: time=1.961s, peak_mem=2.63GB
Step 6: time=0.008s, peak_mem=2.63GB
Step 8: time=0.018s, peak_mem=2.63GB
Step 10: time=2.511s, peak_mem=2.63GB
….
Step 340: time=4.193s, peak_mem=2.63GB
Step 342: time=4.271s, peak_mem=2.63GB
Step 344: time=0.524s, peak_mem=2.63GB
Step 346: time=0.016s, peak_mem=2.63GB
Step 348: time=4.280s, peak_mem=2.63GB
Step 350: time=4.211s, peak_mem=2.63GB
==
avg_step_sec: 2.9172
min_step_sec: 0.0037
max_step_sec: 4.3593
peak_cuda_mem_gb: 2.63
METRICS_JSON: {"avg_step_sec": 2.9172, "min_step_sec": 0.0037, "max_step_sec": 4.3593, "peak_cuda_mem_gb": 2.63}
==
Done in 1289.31 sec
===== torch-benchmark-ts-4 =====
Installing dependencies...
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPSConnectionPool(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/09/c8/844d5518a6aeb4ffdc0cf0cae65ae13dbe5838306728c5c640b5a6e2a0c9/transformers-4.40.0-py3-none-any.whl.metadata
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable.It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Starting training...
GPU: Tesla T4
TOTAL_GPU_MEMORY_GB: 14.56
Всего параметров: 43026442
Обучаемых параметров: 43026442
cuda:0
Starting training...
TRAINING LOOP STARTED — GPU LOAD EXPECTED NOW!
Step 2: time=0.032s, peak_mem=2.63GB
Step 4: time=3.904s, peak_mem=2.63GB
Step 6: time=0.006s, peak_mem=2.63GB
Step 8: time=0.007s, peak_mem=2.63GB
Step 10: time=3.913s, peak_mem=2.63GB
….
Step 342: time=2.813s, peak_mem=2.63GB

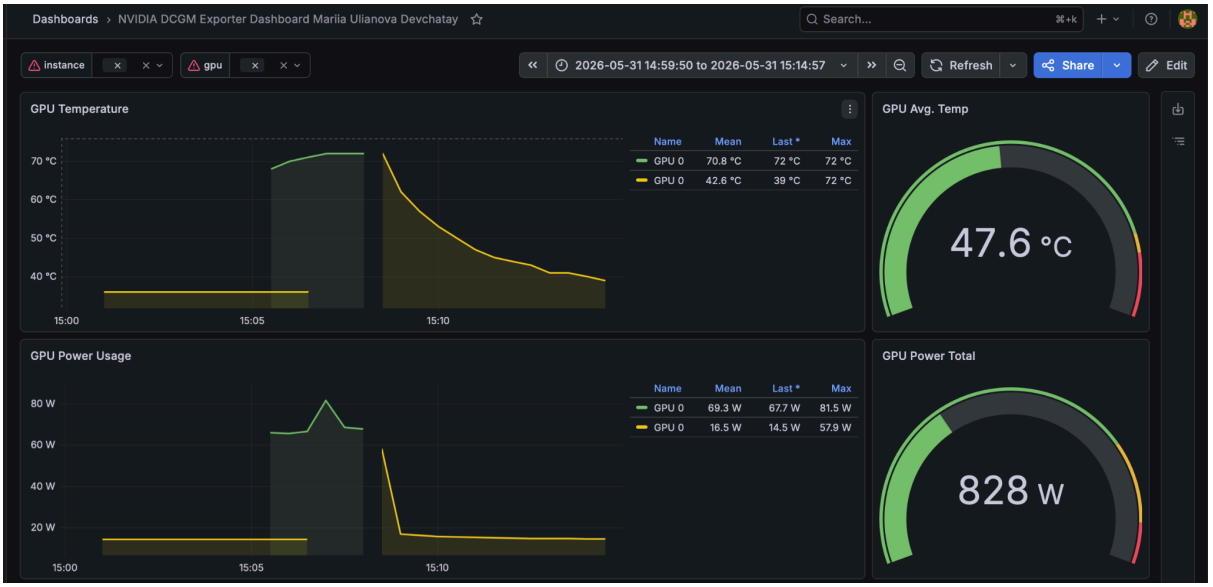
```
Step 344: time=0.253s, peak_mem=2.63GB
Step 346: time=0.016s, peak_mem=2.63GB
Step 348: time=2.152s, peak_mem=2.63GB
Step 350: time=2.167s, peak_mem=2.63GB
============================================================
avg_step_sec: 2.9164
min_step_sec: 0.0039
max_step_sec: 4.3557
peak_cuda_mem_gb: 2.63
METRICS_JSON: {"avg_step_sec": 2.9164, "min_step_sec": 0.0039, "max_step_sec": 4.3557, "peak_cuda_mem_gb": 2.63}
============================================================
Done in 1290.70 sec
devchatay@epds73cmval2ppf6qhpm:~$
```

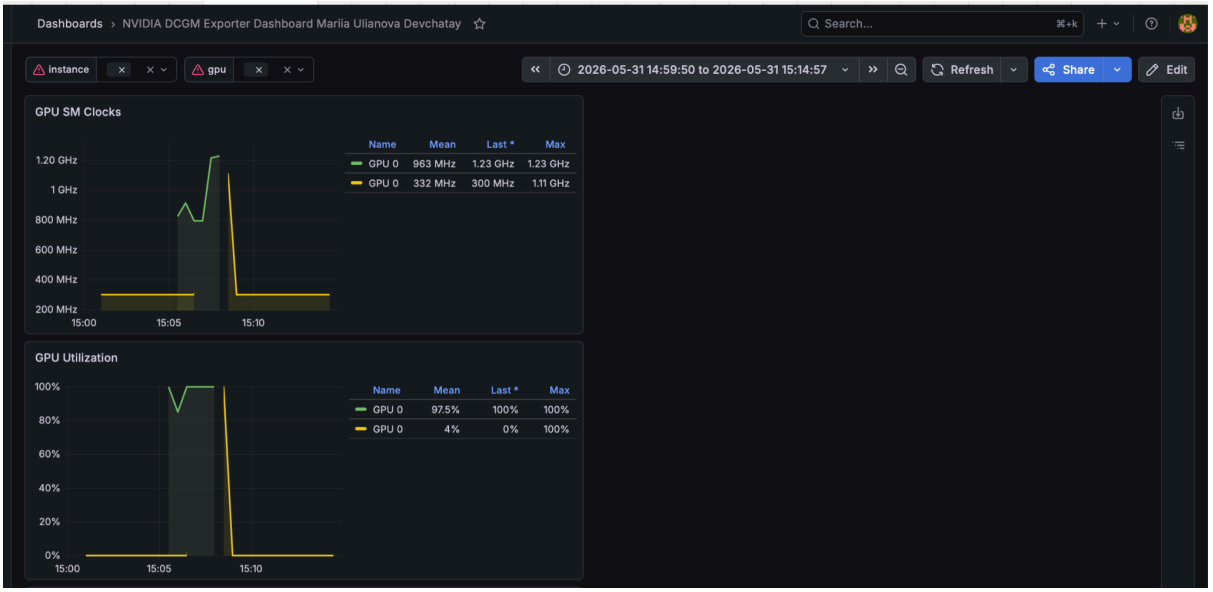
7. **Графики или значения из Grafana по GPU utilization, memory usage и power usage;**

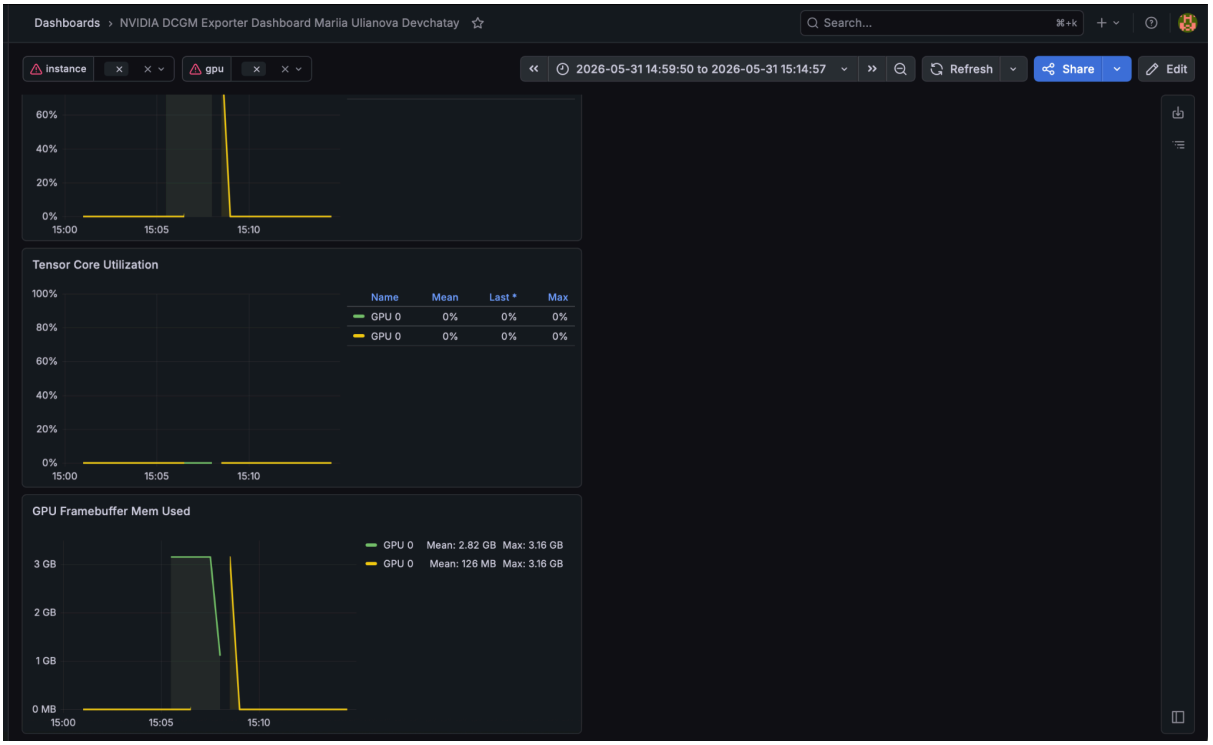
EXCLUSIVE

На этой серии скриншотов показываю, что сначала GPU в ненагруженном состоянии находится, потом нагружается и, когда задача завершается, освобождается. При анализе Grafana в некоторых случаях наблюдалось дублирование графиков для GPU 0, несмотря на запуск только одного workload. Предположительно, это связано не с реальным запуском нескольких процессов, а с особенностями агрегации метрик в связке NVIDIA DCGM Exporter + Prometheus + Grafana. DCGM Exporter может публиковать несколько time series для одной и той же GPU с различными наборами labels (например, pod, container, namespace, instance). При некорректной агрегации в PromQL или выборе слишком общего шаблона Grafana одна физическая GPU отображается несколькими линиями одновременно. Дополнительно причиной может быть одновременное присутствие метрик уровня node и container/pod level metrics. На корректность самого эксперимента это не повлияло, так как значения utilization, memory usage и power usage соответствовали ожидаемому поведению workload.

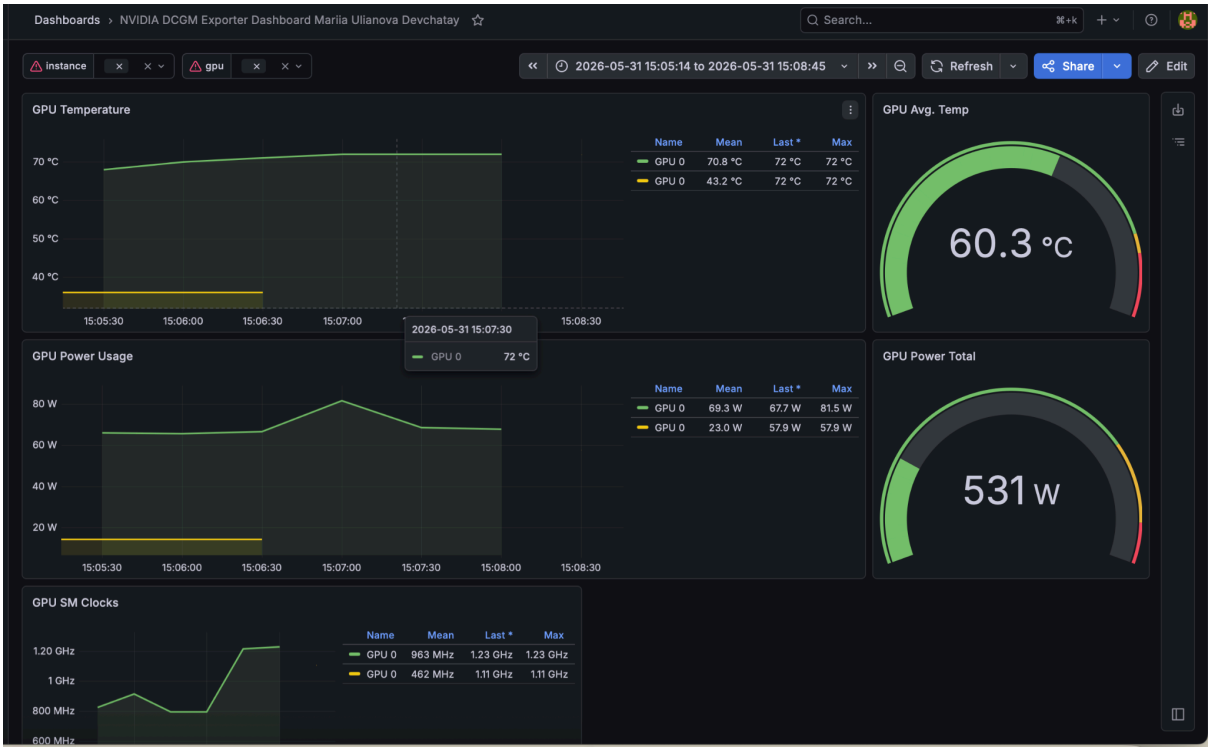
Видно, что достаточно быстро исполнилась задача, температура поднялась с изначальных 36-39 до 47.6, мощность gpu power total небольшая, по памяти используем как раз ровно на одно обучение (2.8-3.6 ГБ).

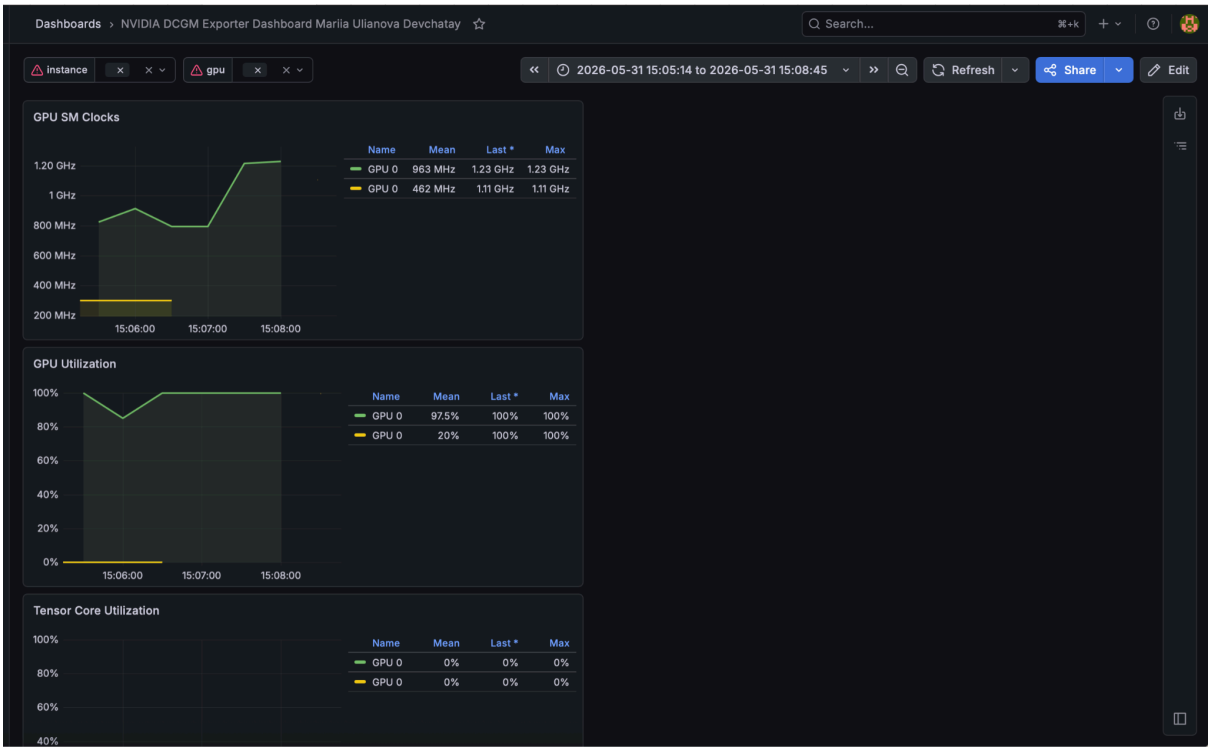

Dashboards › NVIDIA DCGM Exporter Dashboard Mariia Ulianova Devchatay
Search...
instance
gpu
2026-05-31 14:59:50 to 2026-05-31 15:14:57
Refresh
Share
Edit
GPU Temperature
Name Mean Last * Max
GPU 0 70.8 °C 72 °C 72 °C
GPU 0 42.6 °C 39 °C 72 °C
70 °C
60 °C
50 °C
40 °C
15:00
15:05
15:10
GPU Avg. Temp
47.6 °C
GPU Power Usage
Name Mean Last * Max
GPU 0 69.3 W 67.7 W 81.5 W
GPU 0 16.5 W 14.5 W 57.9 W
80 W
60 W
40 W
20 W
15:00
15:05
15:10
GPU Power Total
828 W

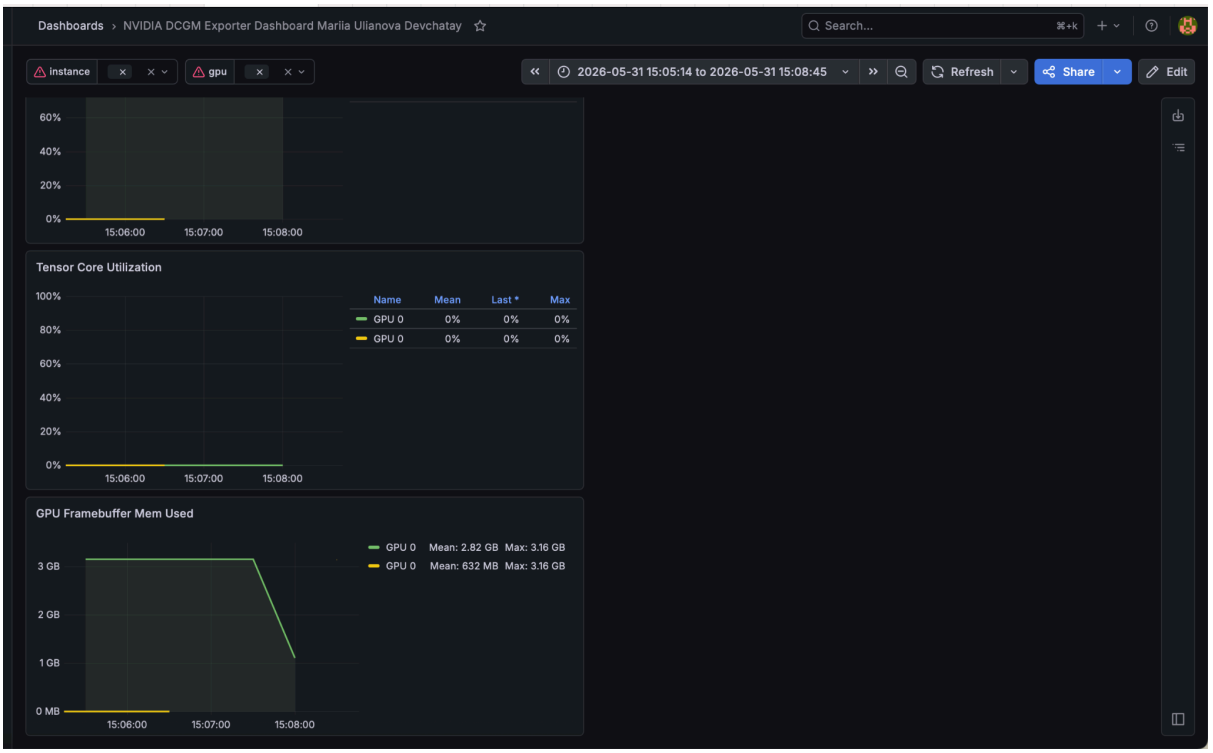

Dashboards › NVIDIA DCGM Exporter Dashboard Mariia Ulianova Devchatay
Search...
instance
gpu
2026-05-31 14:59:50 to 2026-05-31 15:14:57
Refresh
Share
Edit
GPU SM Clocks
Name Mean Last * Max
GPU 0 963 MHz 1.23 GHz 1.23 GHz
GPU 0 332 MHz 300 MHz 1.11 GHz
1.20 GHz
1 GHz
800 MHz
600 MHz
400 MHz
200 MHz
15:00
15:05
15:10
GPU Utilization
Name Mean Last * Max
GPU 0 97.5% 100% 100%
GPU 0 4% 0% 100%
100%
80%
60%
40%
20%
0%
15:00
15:05
15:10



Эта серия скриншотов тоже Exclusive, приблизила конкретное время обучения. Температура 60. Утилизация ~100%.



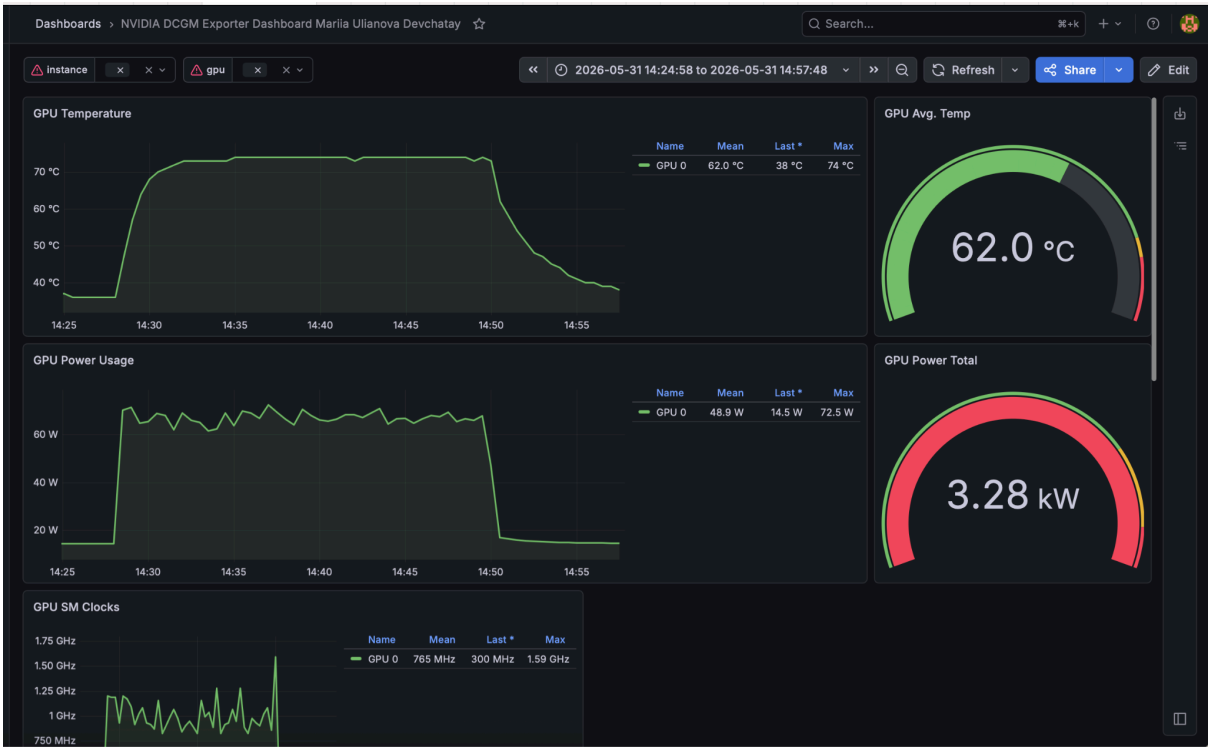


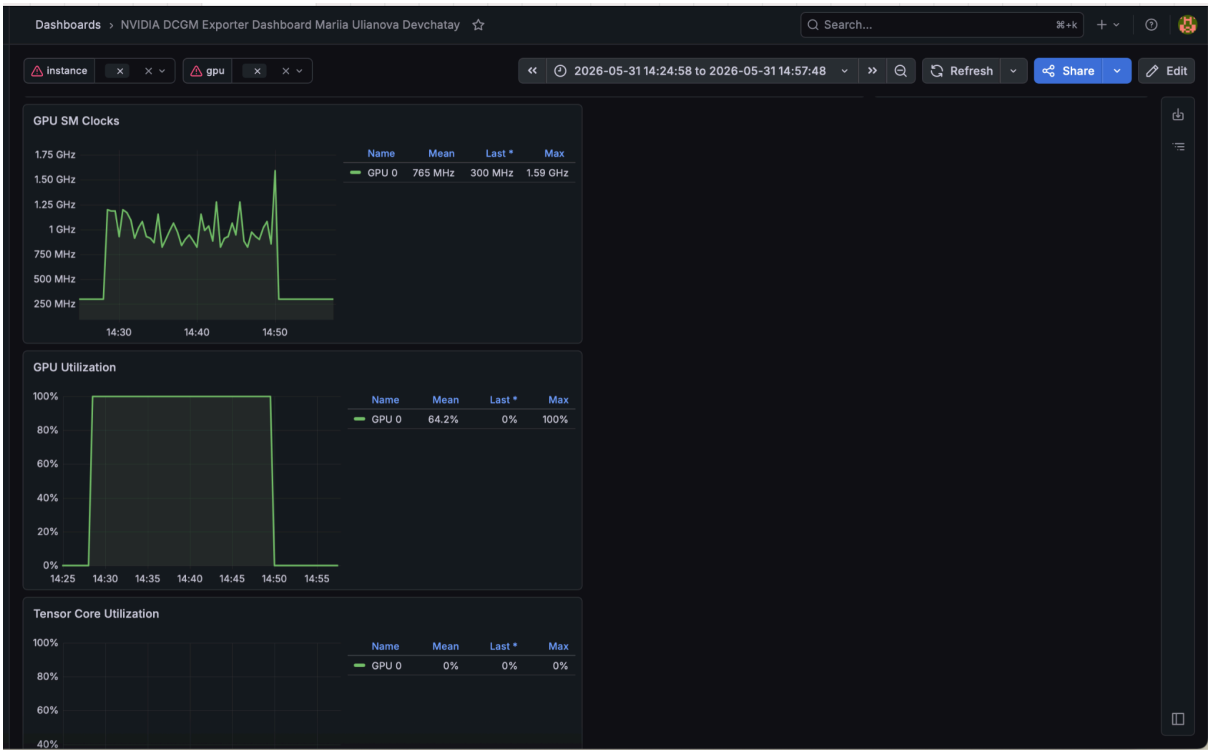


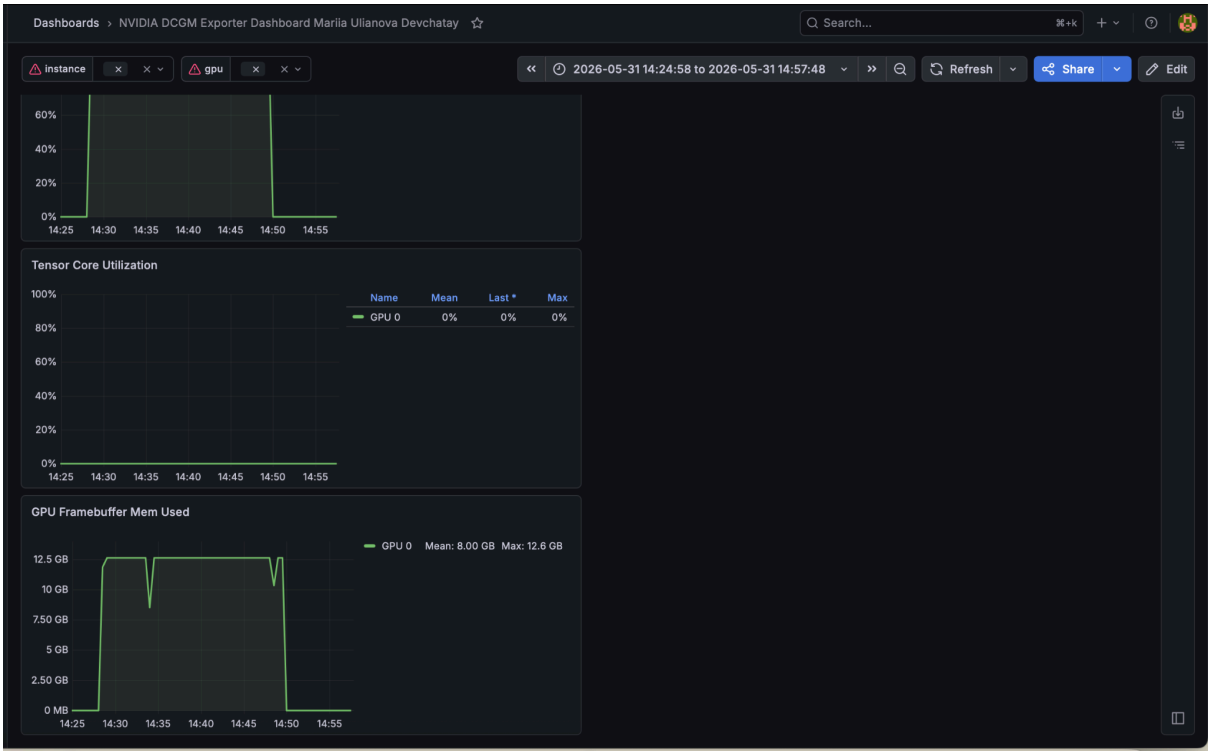
SHARED

Теперь по сравнению с shared. Было 4 пода с одной и той же задачей параллельно. Температура такая же, gpu power total достигла своего

критического значения. Видно, что по времени исполнялось сильно дольше, Exclusive 15.05-15.07, тут ~14.30-14.50, 2-3 минуты и 20 минут. Памяти использовалось тоже в 4 раза больше, максимально достигло 12 ГБ.







8. **Вывод: какой режим подходит для одиночной тяжелой задачи, а какой для нескольких параллельных задач**

Exclusive - одиночная тяжелая задача, так как все ресурсы - видеопамять, мощность и др. направлены только на одну задачу.

Shared - для нескольких параллельных задач. По результатам бенчмарка - время возросло в 4-6 раз, нагруженная тяжелая задача выполнялась параллельно в четырех подах. Уже на такой небольшой (43 млн параметров) сети достигли почти лимита по памяти.

Дополнительно стоит отметить поведение частот SM-ядер. В exclusive частота стабилизируется в районе 1.23 ГГц, что говорит о ровной нагрузке и отсутствии резких переключений контекста. В shared график SM Clocks демонстрирует более выраженные колебания и периодические просадки, что связано с накладными расходами планировщика time-slicing на переключение между

четырьмя подами. Эти микропаузы накапливаются и объясняют, почему среднее время шага выросло не ровно в 4 раза. Также обращает на себя внимание нулевая загрузка Tensor Core - это ожидаемое поведение, так как архитектура модели состоит из стандартных линейных слоёв FP32 и не использует операции смешанной точности, для которых предназначены тензорные ядра. Распределение видеопамяти в shared носит строго изолированный характер, каждый под резервирует около 3 ГБ без динамического перераспределения между процессами, что оставляет менее 2 ГБ свободного буфера на всю карту и создаёт высокий риск out of memory ошибок при любом увеличении batch size или размера модели